Unified Public Transport System (UPTS) - SRS

1. Introduction

1.1 Purpose

This document specifies the requirements for the Unified Public Transport System (UPTS). UPTS is an integrated full stack and Agentic AI system built to support Sri Lanka's public transport network, focused on state and private bus services and the railway system. It is written to satisfy the domain complexity, technology stack, agent design, and cross platform workflow requirements set out in the SE3090 Assignment 1 specification.

1.2 Background and Motivation

Sri Lanka is currently modernizing its public transport sector, including GPS based bus tracking, digital ticketing, and smart card initiatives across state and private bus operators and the railway. However, this modernization is fragmented across separate operators and systems, and there is no single citizen-facing platform that unifies trip planning, booking, payment, and safety reporting across bus and rail services. UPTS addresses this gap as a coordination layer that sits on top of existing bus and rail operations, in line with the direction the government is already taking with its digital public transport initiatives.

1.3 Scope

UPTS consists of the following parts:

- A React web application used by Transit Administrators to manage fleets, routes, and to monitor and approve Agentic AI decisions.
- A Flutter mobile application used by Commuters to search, book, and track trips, and by Drivers or Operators to manage their assigned trips.
- A shared ASP.NET Core Web API backend, using Entity Framework Core and PostgreSQL as the database.
- An Agentic AI subsystem made up of four distinct agents that plan, analyze, execute, and validate the trip booking process.
- ThunderID as the Identity Provider (IdP) for authentication and role based access control across both the React and Flutter clients.

UPTS covers bus and rail based public transport only. Informal or unregulated transport modes such as three wheelers are outside the scope of this system, since the assignment domain is intentionally centered on the formal public transport network that the government is currently digitizing.

1.4 Definitions and Abbreviations

Term	Meaning
IdP	Identity Provider
DPI	Digital Public Infrastructure
Agent	A distinct AI worker with a defined responsibility, a clear input and output contract, and a controlled set of tools it is allowed to use
High impact action	An action that requires human approval before it is executed, such as a fare override
OIDC	OpenID Connect, the identity protocol used by ThunderID

2. Overall Description

2.1 Product Perspective

UPTS is a new, standalone system made up of four business components, with one component owned by each group member. All four components share a single identity system through ThunderID, a shared permission model, and shared business rules across the React and Flutter clients. This satisfies the assignment's requirement for one coherent integrated system rather than four disconnected mini projects.

2.2 User Roles

Role	Responsibilities	Primary Client
Commuter	Search bus and train routes, book trips, make payments, track vehicles in real time, report incidents	Flutter
Driver / Operator	View and accept assigned trips, update trip and vehicle status, report incidents from the field	Flutter / React
Transit Admin	Manage fleets and routes, monitor Agentic AI workflows, approve or reject high impact actions, view operational reports	Admin

2.3 Assumptions and Constraints

- Vehicle location is simulated or crowd sourced for the purposes of this assignment, since live hardware GPS integration with real operators is out of scope.
- Payment processing uses a sandbox or mock payment gateway rather than a live financial provider.
- Authentication, identity, and role management are fully delegated to ThunderID using OIDC and OAuth2. The ASP.NET Core API never stores or manages raw passwords.
- Only free or institution provided services are used, in line with the assignment's cost constraint.

3. Domain Complexity Compliance

This table maps UPTS directly to the domain complexity requirements in the assignment specification.

Requirement	How UPTS Satisfies It
3 or more user roles with distinct permissions	Commuter, Driver or Operator, and Transit Admin
4 major business components	Trip Booking and Ticketing, Fleet and Route Management, Wallet and Payment, Incident and Safety Reporting
CRUD plus status workflows, search, filter, sort, pagination, and reporting	Implemented individually in each component, described in Section 4
Distinct purposes for React and Flutter	React serves administrative and monitoring functions, Flutter serves commuter and driver facing operations
At least one third party integration	A maps and routing API, plus a notification API for SMS or email
One complete cross platform workflow	Trip booking through to Agentic AI processing, admin approval, and status return, described in Section 6

4. Business Components

4.1 Component A: Trip Booking and Ticketing

- Search available bus and train trips by origin, destination, and time.
- Book a seat, generate a QR based boarding pass.
- View booking history and status, such as pending, confirmed, in transit, completed, or cancelled.
- Cancel or reschedule a booking within defined policy rules.
- Business specific operation: dynamic fare estimation based on distance, current demand, and route or transport type.

4.2 Component B: Fleet and Route Management

- Register and manage buses, trains, and drivers.
- Define routes, stops, schedules, and seating capacity.
- Assign drivers and vehicles to scheduled trips.
- Business specific operation: route performance analytics, including on time rate and average occupancy per route.

4.3 Component C: Wallet and Payment Management

- Wallet top up and balance tracking for each commuter.
- Fare deduction on trip completion.
- Payment history and refund handling.
- Business specific operation: driver and operator payout reconciliation across completed trips.

4.4 Component D: Incident and Safety Reporting

- Commuters and drivers can report delays, hazards, or safety incidents.
- SLA based tracking of how quickly incidents are resolved.
- Flagging of recurring issues tied to a specific driver or vehicle.
- Business specific operation: incident trend analytics that feed directly into the Compliance and Validation Agent described below.

Each component exposes at least four meaningful API endpoints and at least one business specific operation beyond basic CRUD, in line with the assignment's minimum expectations.

5. Agentic AI Subsystem

UPTS uses four distinct agents, each with a clearly separated responsibility. Together they form one complete, assessed workflow rather than four independent chatbots. Each agent maps to one of the four required agent roles described in the assignment: planning and coordination, domain analysis, action and tool use, and validation and safety.

5.1 The Four Agents Explained

1. Trip Planning Agent (Planning and Coordination) This agent receives the commuter's original request, for example "I want to travel from Kurunegala to Colombo at 8 AM tomorrow." It does not call any external tool. Its job is only to turn that request into a clear, ordered plan: find matching routes, estimate the fare, check seat or capacity availability, then propose a booking. This plan is then handed to the next agent.

2. Route and Demand Analysis Agent (Domain Analysis) This agent takes the plan from the Trip Planning Agent and analyzes it against real operational data. It looks at available route options between the origin and destination, checks current demand and crowding levels on those routes, and confirms that a suitable driver and vehicle are actually eligible and available for that route and time. It selects the best matching route and vehicle before passing the decision forward.

3. Booking and Dispatch Agent (Action and Tool Use) This is the only agent that actually calls external tools and performs real actions. It uses an allow listed set of tools: a maps and routing API to confirm distance and travel time, a fare calculator, a booking creation tool that writes the booking into the database, and a notification dispatcher that sends the commuter a confirmation. It executes the booking based on what the previous two agents decided.

4. Compliance and Validation Agent (Validation and Safety) This agent checks the outcome of the booking against fixed business rules before it is finalized. It confirms the fare falls within an expected range, confirms the vehicle is not over capacity, and looks for anomalies such as an unusually high fare or a last minute driver reassignment. If something falls outside normal rules, this agent pauses the workflow and sends it to a Transit Admin for human approval through the React dashboard, rather than letting the action complete automatically.

5.2 Shared Workflow State

Each workflow has a unique ID and stores its objective, its plan, each completed step, the results of any tool calls, the validation outcome, the approval status, and the final result. All of this is persisted in PostgreSQL for auditability. No passwords, tokens, or raw model reasoning are stored.

5.3 Human Approval

Any flagged fare anomaly or unusual driver reassignment pauses the workflow. A Transit Admin must approve, reject, or request a revision through the React dashboard before the workflow can continue.

5.4 Observability and Security

Every tool call, timing, validation result, retry, and approval decision is logged and visible to Transit Admins in React. Agents use role based access, validate their own inputs and outputs, apply timeouts and retry limits, and are restricted to a fixed, allow listed set of tools with least privilege access.

6. Cross Platform Workflow

This is the minimum assessed end to end workflow that ties every layer of the system together.

1. A Commuter submits a trip request from the Flutter app, authenticated through ThunderID.
2. The request triggers the Agentic AI workflow through the ASP.NET Core backend, and the workflow state is persisted in PostgreSQL.
3. The four agents plan, analyze, execute, and validate the booking. If an anomaly is flagged, such as a fare override, the workflow pauses for approval.
4. A Transit Admin reviews and approves or rejects the flagged action from the React dashboard, also authenticated through ThunderID.
5. The final booking status is returned to the Commuter in the Flutter app in real time.

7. External Interface Requirements

7.1 React Web Application

- Login through ThunderID using OIDC, protected routes, and role based navigation.
- CRUD interfaces for fleet, route, and incident data, with search, filter, sort, and pagination.
- An Agentic AI workflow monitoring dashboard with approve, reject, and request revision controls.
- Reporting and analytics views covering occupancy, incidents, and fare trends.

7.2 Flutter Mobile Application

- Login and registration through ThunderID, with secure token storage and protected screens.
- Trip search, booking, QR boarding pass display, and live vehicle tracking on a map.
- A driver view for accepting trips and updating trip or vehicle status.
- Push notifications for booking confirmations and trip updates.
- Device features used: GPS and maps for live tracking, and QR scanning for boarding verification.

7.3 Third Party Integrations

- Maps and Routing API, used for distance estimation, travel time estimation, and route rendering.
- Notification API (SMS or email), used for booking confirmations, delay alerts, and approval notices.
- ThunderID, used for identity, authentication, and role or permission management through OIDC, OAuth2, and JWT issuance.

8. Data Requirements

High level entities: User (linked through the ThunderID subject ID), Vehicle, Driver, Route, Trip, Booking, Wallet, Transaction, Incident, AgentWorkflow, AgentStep, ApprovalRequest.

All entities include CreatedAt and UpdatedAt audit fields, along with appropriate primary keys, foreign keys, constraints, and indexes. A complete ER diagram will be produced during the database design phase.

9. Non Functional Requirements

- Security: ThunderID issued JWTs are validated by ASP.NET Core, role based authorization is enforced on every endpoint, and all inputs to the API and to agent tool calls are validated.
- Performance: Paginated queries, asynchronous API operations, and acceptable latency for the Agentic AI workflow.
- Reliability: Safe failure behavior when an agent or tool call fails, with defined retry limits and timeouts.
- Auditability: Every agent action and every admin approval decision is logged.
- Usability: Both clients handle loading, empty, success, and error states clearly and consistently.

10. Technology Stack

Layer	Technology
Backend	Entity Framework Core, PostgreSQL provider
Database	PostgreSQL
Identity Provider	ThunderID, using OIDC and OAuth2 for authentication, JWT issuance, and role management
Web Frontend	React, using functional components, hooks, React Router, and a state management approach to be justified in the ADR
Mobile Frontend	Flutter and Dart, with a state management approach to be justified in the ADR
Agentic AI Orchestration	IDK ABOUT THIS SHIT
Third Party Services	Maps and routing API, SMS or email notification API

Version Control and CI	Git and GitHub, with GitHub Actions running backend tests on every push and pull request to main
Deployment	A cloud platform for the API, PostgreSQL, and React frontend, and a runnable APK for Flutter

11. Member Task Allocation

Each student owns one business component end to end, meaning backend, database, React, Flutter, tests, Git evidence, and documentation for that component, plus one distinct Agentic AI contribution. No student is assigned to a project management only, testing only, or documentation only role.

Student 1: Trip Booking and Ticketing (Component A)

- Backend: Trip search, booking creation, QR boarding pass generation, booking status and history endpoints.
- Database: Trip and Booking tables, with relationships to Route and Vehicle.
- React: Admin trip and booking oversight views, with search, filter, sort, and pagination.
- Flutter: Commuter trip search and booking flow, QR pass display, booking history screen.
- Agentic AI: Trip Planning Agent, responsible for the plan generation logic that starts each workflow.
- Testing: Unit and integration tests for booking endpoints, plus Flutter widget tests for the booking flow.

Student 2: Fleet and Route Management (Component B)

- Backend: Vehicle and driver registration, route, stop, and schedule CRUD, trip assignment endpoints.
- Database: Vehicle, Driver, and Route tables and their relationships.
- React: Fleet and route management dashboard, route performance analytics view.
- Flutter: Driver view for accepting trips and updating vehicle or trip status.
- Agentic AI: Route and Demand Analysis Agent, responsible for route and demand scoring and eligibility checks.
- Testing: Backend integration tests for fleet and route logic, React component tests for the dashboard.

Student 3: Wallet and Payment Management (Component C)

- Backend: Wallet top up, fare deduction, transaction history, refund and payout endpoints.
- Database: Wallet and Transaction tables, including constraints and transaction handling.
- React: Payment and reporting dashboard, payout reconciliation view.
- Flutter: Commuter wallet screen, payment history, top up flow.

- Agentic AI: Booking and Dispatch Agent, responsible for all tool calls, including the maps API, fare calculator, booking creation tool, and notification dispatcher.
- Testing: Transaction and constraint tests, API integration tests, Flutter form validation tests.

Student 4: Incident and Safety Reporting (Component D)

- Backend: Incident CRUD, SLA tracking, incident trend analytics endpoints.
- Database: Incident table, with relationships to Trip, Driver, and Vehicle.
- React: Incident dashboard, plus the Agentic AI workflow monitoring and approve, reject, revise interface.
- Flutter: Incident reporting form for commuters and drivers, with status tracking.
- Agentic AI: Compliance and Validation Agent, responsible for business rule validation, anomaly flagging, and human approval gating.
- Testing: Agent evaluation golden case tests, the end to end cross platform workflow test, and performance tests.

Shared Responsibilities (All Members)

- ThunderID integration on each member's own endpoints and screens.
- Participation in GitHub Actions CI, code review, and pull requests.
- Contribution to the consolidated README, the ADR entries relevant to their component, and their individual report section.

12. Architecture Decision Record (ADR) Topics To Cover

1. State management approach in React.
2. State management approach in Flutter.
3. Agentic AI framework and orchestration method.
4. Database schema strategy for Agentic AI workflow state.
5. Identity provider choice, ThunderID, including context, alternatives considered, and consequences.
6. Cloud deployment platform.

13. Out of Scope

- Real hardware GPS integration with live bus and rail operators.
- Integration with a national digital ID system for citizen level identity verification.
- Multi operator settlement and revenue sharing automation across private bus companies.